Java Programming Assessment Solutions

Section A: Short Questions

a) What is the use of finally?
The `finally` block is used in exception handling to execute crucial cleanup code (like closing files or database connections) regardless of whether an exception is thrown or caught.

b) What is a static variable?
A variable declared with the `static` keyword belongs to the class itself rather than to any specific object instance, meaning all objects of that class share a single copy of the variable.

c) Which keyword is used to inherit a class in Java?
The `extends` keyword.

d) What is the default value of a boolean instance variable in Java?
`false`.

e) What is constructor in Java?
A special block of code with the same name as the class that is called automatically when an object is instantiated, primarily used to initialize the object's state.

f) Which method is used to compare the contents of two strings in Java?
The `equals()` method (or `equalsIgnoreCase()` to ignore case).

g) What is bytecode in Java?
Platform-independent intermediate code generated by the Java compiler (`javac`) from source code, which is then executed by the Java Virtual Machine (JVM).

h) What is the purpose of Thread.sleep()?
It pauses (suspends) the execution of the currently running thread for a specified amount of time in milliseconds.

i) Define polymorphism.
An object-oriented programming concept that allows a single action to be performed in different ways. In Java, it is achieved through method overloading (compile-time) and overriding (run-time).

j) What is the use of the FileWriter class?
It is a character-oriented class used to write character data (text) directly to a file.

k) Which method returns the length of a string?
The `length()` method.

l) What is the role of JVM?
The Java Virtual Machine (JVM) provides the runtime environment to execute Java bytecode, translating it into machine-specific code so the program can run on any underlying operating system.

Section B: Medium Questions

2. Difference between `this` and `super`

Feature	`this` Keyword	`super` Keyword
Reference	Refers to the current class instance.	Refers to the immediate parent class instance.
Usage (Vars/Methods)	Used to access current class variables/methods.	Used to access parent class variables/methods.
Constructors	`this()` invokes a constructor of the same class.	`super()` invokes a constructor of the parent class.

3. Difference between `throw` and `throws`

Feature	`throw` Keyword	`throws` Keyword
Purpose	Used to explicitly throw an exception from within a method.	Used in a method signature to declare exceptions it might throw.
Followed by	An instance of an Exception (e.g., `new Exception()`).	Names of Exception classes (e.g., `IOException`).
Location	Inside the method body.	With the method signature.

4. Creating Threads (Runnable vs Thread)

```
// Thread 1: Extending Thread class
class MyThreadClass extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Thread Class: " + i);
        }
    }
}

// Thread 2: Implementing Runnable interface
class MyRunnableInterface implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Runnable Interface: " + i);
        }
    }
}

public class ThreadDemonstration {
    public static void main(String[] args) {
        MyThreadClass t1 = new MyThreadClass();
        t1.start();

        Thread t2 = new Thread(new MyRunnableInterface());
        t2.start();
    }
}
```

5. Method Overloading vs Method Overriding

Feature	Method Overloading	Method Overriding
Location	Occurs within the same class.	Occurs between parent and child classes (IS-A).
Polymorphism	Compile-time (Static) Polymorphism.	Run-time (Dynamic) Polymorphism.
Parameters	Parameter lists MUST be different.	Parameter lists MUST be exactly the same.

Section C: Long Questions

6. Inheritance Types, Interfaces, and Abstract Classes

Inheritance Types:

- **Single:** Subclass inherits from one superclass. `class Dog extends Animal {}`
- **Multilevel:** Subclass inherits from a child class. `class Puppy extends Dog {}`
- **Hierarchical:** Multiple subclasses inherit from one superclass. `class Cat extends Animal {}`

Abstract Class vs Interface:

An **Abstract Class** (using `abstract` keyword) can have both abstract and concrete methods. It provides a core identity. An **Interface** defines a strict contract using only abstract methods (prior to Java 8) and public static final constants. A class can implement multiple interfaces but extend only one abstract class.

7. Exceptions and Thread Life Cycle

Exceptions:

- **Checked:** Verified at compile-time (e.g., `IOException`).
- **Unchecked:** Occur at runtime, unchecked by compiler (e.g., `NullPointerException`).
- **User-Defined:** Custom exceptions extending `Exception` or `RuntimeException`.

Thread Life Cycle:

```
[ NEW ] -> [ RUNNABLE ] <-> [ BLOCKED/WAITING ]
               |
               v
         [ TERMINATED ]
```

8. 2D Array Extremes and Access Modifiers

```
public class MatrixExtremes {
    public static void main(String[] args) {
        int[][] matrix = { { 12, 45, 2 }, { 99, -5, 34 } };
        int min = matrix[0][0], max = matrix[0][0];

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                if (matrix[i][j] > max) max = matrix[i][j];
                if (matrix[i][j] < min) min = matrix[i][j];
            }
        }
        System.out.println("Max: " + max + ", Min: " + min);
    }
}
```

Access Modifiers:

- **Private:** Accessible only within the same class.
- **Default:** Accessible only within the same package.
- **Protected:** Accessible within the same package and subclasses everywhere.
- **Public:** Accessible everywhere.

9. Final Keyword, Java Features, and JVM Architecture

Final Keyword: Used to restrict modifications. A final variable becomes a constant, a final method cannot be overridden, and a final class cannot be inherited.

Java Features: Object-Oriented, Platform-Independent (WORA), Secure, Robust, Multithreaded.

JVM Architecture:

1. **Class Loader Subsystem:** Loads, links, and initializes `.class` files.
2. **Runtime Data Areas:** Memory allocations (Method Area, Heap, Java Stacks, PC Registers, Native Stacks).
3. **Execution Engine:** Executes bytecode via Interpreter, JIT Compiler, and Garbage Collector.

10. Exception Keywords and String Class Methods

Keywords: `try` (wraps risky code), `catch` (handles errors), `finally` (cleanup code), `throw` (triggers custom exception), `throws` (declares potential exceptions in method signature).

String Class (Immutable): Commonly used methods include:

1. `length()` : Returns character count.
2. `charAt(int index)` : Returns character at specific index.
3. `substring(int beginIndex)` : Extracts part of the string.
4. `toUpperCase()` : Converts string to uppercase.
5. `equals(Object another)` : Compares content for exact equality.